



实验名称：SQL 基础操作实验

作者：郭威威

学号：2414308

指导老师：李旭东

日期：2025 年 3 月 30 日

目录

实验目的	2
实验内容与步骤	2
QUIZ 1 :	2
QUIZ 2 :	4
1. 分组统计学院教师数，排序。	4
2. 统计多教师授课的课程，取课程名前 5 字符。 ...	5
3. 通过教师 ID 找学院，统计该学院学生数。	6
4. 筛选成绩等级，关联学生和学院。	7
5. 计算工资均值和中位数，注意中位数在 SQL 中的实现。	8
6. 分析两个 SQL 语句的区别，主要看 JOIN 条件位置。	10
实验结论	11

实验目的

掌握 SQL 语言的基本语法，熟悉数据库的创建、表结构设计、数据插入与查询操作，理解关系型数据库的逻辑结构。

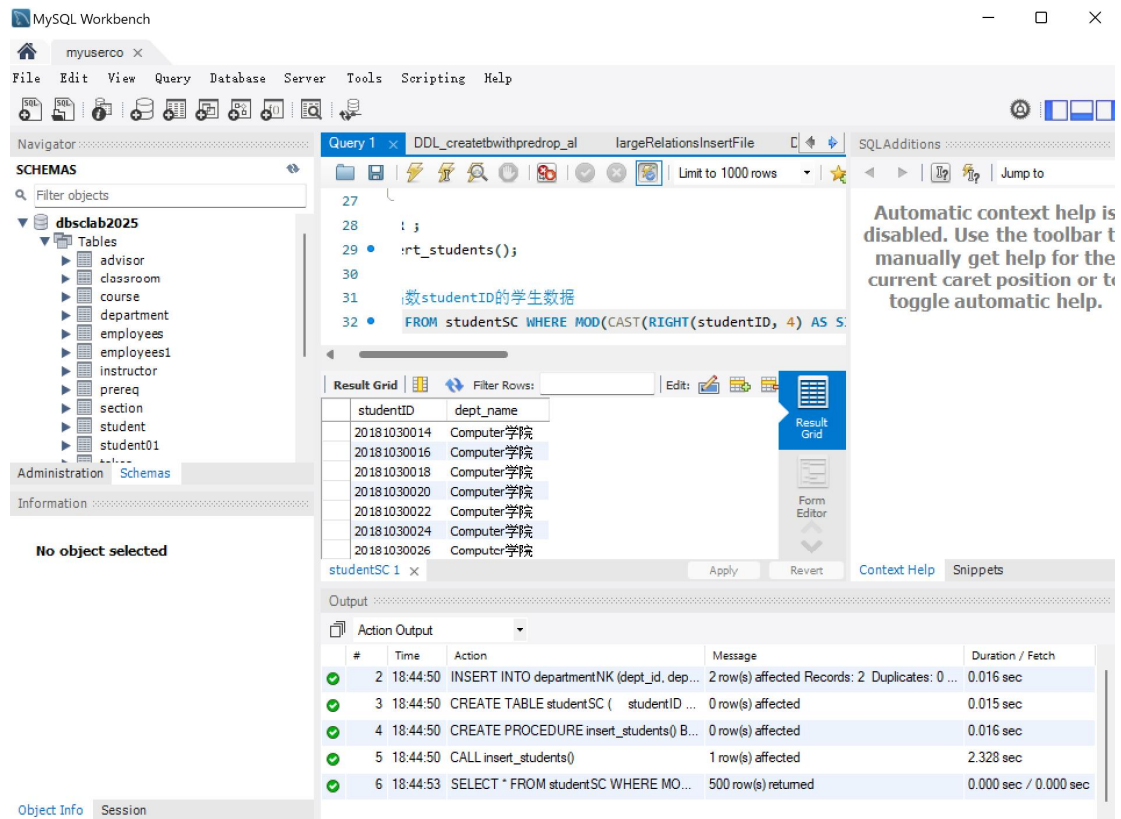
具体包括：

- 基本查询与自然连接
- 重命名操作与字符串处理
- 元组排序与集合操作
- 聚合函数与嵌套子查询

实验内容与步骤

QUIZ 1 :

1. 创建 departmentNK 表，假设字段合理，如学院相关。
2. 插入学院数据。
3. 创建 studentSC 表，包含 studentID 等字段。
4. 插入 1000 条数据，ID 范围正确，学院为 Computer 学院。
5. 查询偶数 ID 的学生。



代码如下 1

```

sql
-- 创建 departmentNK 表
CREATE TABLE departmentNK (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

-- 插入学院数据
INSERT INTO departmentNK (dept_id, dept_name) VALUES
(1, 'Computer 学院'), (2, '其他学院');

-- 创建 studentSC 表
CREATE TABLE studentSC (
    studentID VARCHAR(20) PRIMARY KEY,
    dept_name VARCHAR(50)
);

-- 插入 1000 条学生数据
DELIMITER $$
CREATE PROCEDURE insert_students()
BEGIN
    DECLARE i INT DEFAULT 1;

```

```

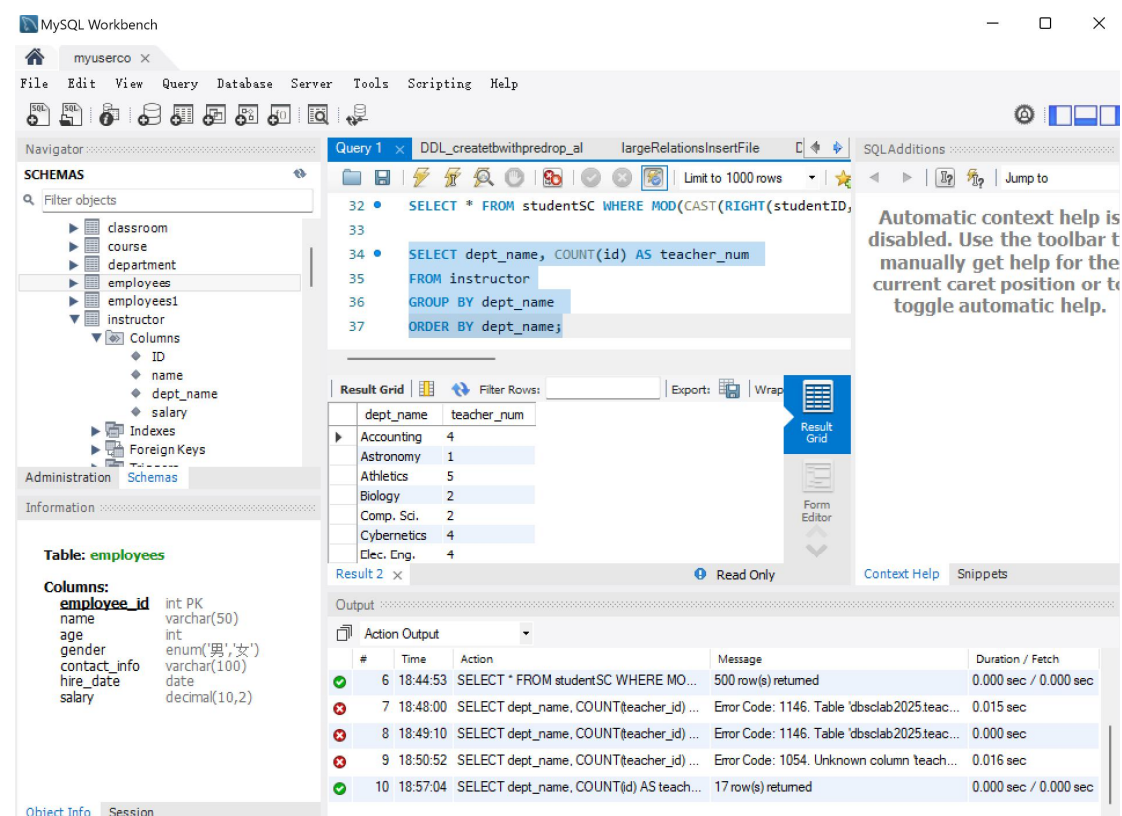
WHILE i <= 1000 DO
    INSERT INTO studentSC (studentID, dept_name)
    VALUES (CONCAT('2018103', LPAD(i, 4, '0')), 'Computer 学院');
    SET i = i + 1;
END WHILE;
END $$
DELIMITER ;
CALL insert_students();

-- 显示偶数 studentID 的学生数据
SELECT * FROM studentSC WHERE MOD(CAST(RIGHT(studentID, 4) AS SIGNED), 2) = 0;

```

QUIZ 2 :

1. 分组统计学院教师数，排序。



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employees' selected. The main editor shows a query in 'Query 1' window:

```

SELECT * FROM studentSC WHERE MOD(CAST(RIGHT(studentID, 4) AS SIGNED), 2) = 0;
SELECT dept_name, COUNT(id) AS teacher_num
FROM instructor
GROUP BY dept_name
ORDER BY dept_name;

```

The 'Result Grid' shows the output of the second query:

dept_name	teacher_num
Accounting	4
Astronomy	1
Athletics	5
Biology	2
Comp. Sci.	2
Cybernetics	4
Elec. Eng.	4

The 'Output' window at the bottom shows the execution log:

#	Time	Action	Message	Duration / Fetch
6	18:44:53	SELECT * FROM studentSC WHERE MO...	500 row(s) returned	0.000 sec / 0.000 sec
7	18:48:00	SELECT dept_name, COUNT(teacher_id) ...	Error Code: 1146. Table 'db:slab2025.teac...	0.015 sec
8	18:49:10	SELECT dept_name, COUNT(teacher_id) ...	Error Code: 1146. Table 'db:slab2025.teac...	0.000 sec
9	18:50:52	SELECT dept_name, COUNT(teacher_id) ...	Error Code: 1054. Unknown column teach...	0.016 sec
10	18:57:04	SELECT dept_name, COUNT(id) AS teach...	17 row(s) returned	0.000 sec / 0.000 sec

代码

```

sql
SELECT dept_name, COUNT(id) AS teacher_num
FROM instructor

```

```
GROUP BY dept_name
ORDER BY dept_name;
```

2. 统计多教师授课的课程，取课程名前 5 字符。

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'course' and 'instructor'. The 'Information' pane shows details for the 'course' table, including columns: 'course_id' (varchar(8) PK), 'title' (varchar(50)), 'dept_name' (varchar(20)), and 'credits' (decimal(2,0)).

The main query editor shows a query (Query 1) with the following SQL code:

```
38
39 • SELECT course.course_id, LEFT(title, 5) AS short_name,
40 FROM course,takes
41 WHERE course.course_id IN (
42 SELECT course.course_id FROM instructor GROUP BY co
43 );
```

The 'Result Grid' shows the results of the query, with columns 'course_id', 'short_name', and 'semester'. The results are as follows:

course_id	short_name	semester
998	Immun	Fall
991	Trans	Fall
984	Music	Fall
983	Virol	Fall
974	Astro	Fall
972	Greek	Fall
969	The M	Fall

The 'Output' pane at the bottom shows the execution log, including the query execution time and the number of rows returned.

代码

```
sql
-- 创建 departmentNK 表
CREATE TABLE departmentNK (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

-- 插入学院数据
INSERT INTO departmentNK (dept_id, dept_name) VALUES
(1, 'Computer 学院'), (2, '其他学院');

-- 创建 studentSC 表
CREATE TABLE studentSC (
    studentID VARCHAR(20) PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

```

-- 插入 1000 条学生数据
DELIMITER $$
CREATE PROCEDURE insert_students()
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 1000 DO
        INSERT INTO studentSC (studentID, dept_name)
        VALUES (CONCAT('2018103', LPAD(i, 4, '0')), 'Computer 学院
');
        SET i = i + 1;
    END WHILE;
END $$
DELIMITER ;
CALL insert_students();

-- 显示偶数 studentID 的学生数据
SELECT * FROM studentSC WHERE MOD(CAST(RIGHT(studentID, 4) AS
SIGNED), 2) = 0;

SELECT dept_name, COUNT(id) AS teacher_num
FROM instructor
GROUP BY dept_name
ORDER BY dept_name;

SELECT course.course_id, LEFT(title, 5) AS short_name, semester
FROM course,takes
WHERE course.course_id IN (
    SELECT course.course_id FROM instructor GROUP BY
course.course_id HAVING COUNT(id) > 1
);

```

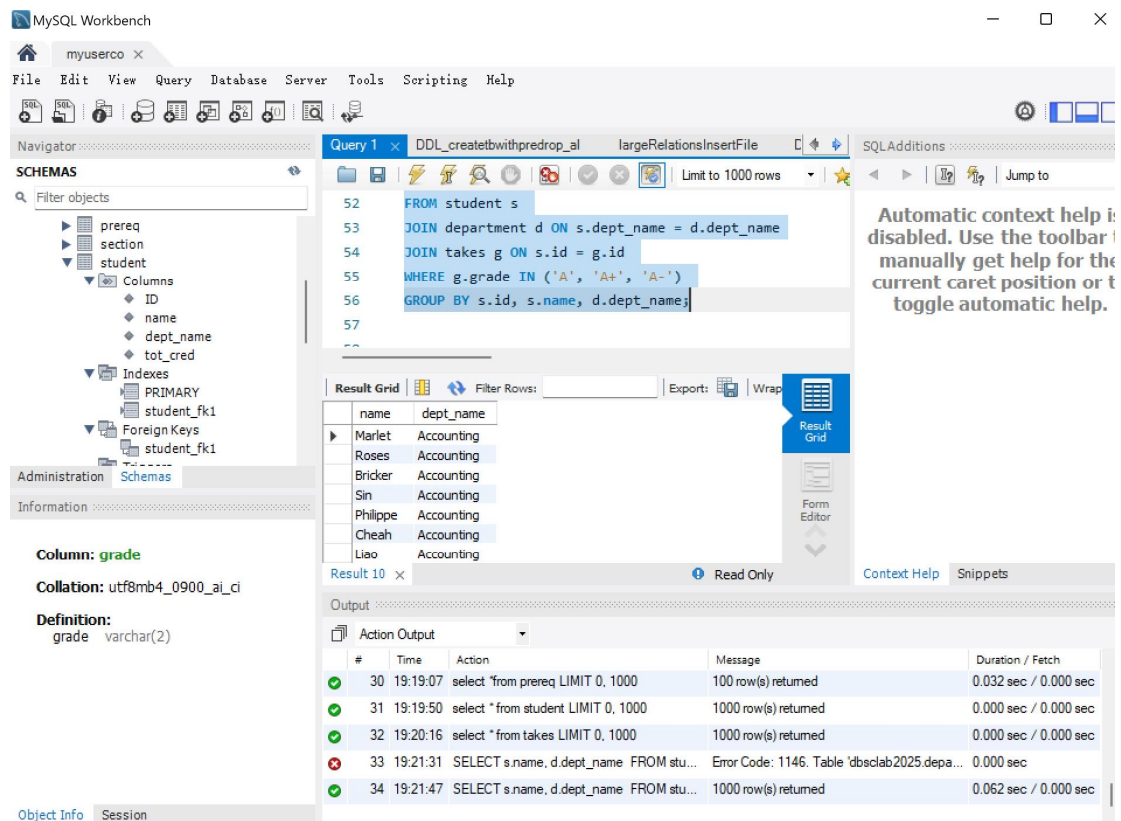
3. 通过教师 ID 找学院，统计该学院学生数。

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

代码

```
sql
SELECT COUNT(s.id)
FROM student s
JOIN department d ON s.dept_name = d.dept_name
JOIN instructor t ON d.dept_name = t.dept_name
WHERE t.id = 14365;
```

4. 筛选成绩等级，关联学生和学院。



代码

```

sql
SELECT s.name, d.dept_name
FROM student s
JOIN department d ON s.dept_name = d.dept_name
JOIN takes g ON s.id = g.id
WHERE g.grade IN ('A', 'A+', 'A-')
GROUP BY s.id, s.name, d.dept_name;

```

5. 计算工资均值和中位数，注意中位数在 **SQL** 中的实现。

MySQL Workbench

myuserco x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- dept_name
- tot_cred
- Indexes
 - PRIMARY
 - student_fk1
- Foreign Keys
 - student_fk1
- Triggers
- student01
- takes
 - Columns
 - ID
 - course_id

Administration Schemas

Information

Column: grade

Collation: utf8mb4_0900_ai_ci

Definition:

grade varchar(2)

Object Info Session

Query 1 x DDL_createtbwithpredrop_al largeRelationsInsertFile

Limit to 1000 rows

SQLAdditions

Jump to

53 JOIN department d ON s.dept_name = d.dept_name

54 JOIN takes g ON s.id = g.id

55 WHERE g.grade IN ('A', 'A+', 'A-')

56 GROUP BY s.id, s.name, d.dept_name;

57

58 SELECT AVG(salary) AS mean_salary FROM instructor;

Result Grid

mean_salary
77600.188200

Result 11 x Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
32	19:20:16	select * from takes LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
33	19:21:31	SELECT s.name, d.dept_name FROM stu...	Error Code: 1146. Table 'dbaclab2025.depa...	0.000 sec
34	19:21:47	SELECT s.name, d.dept_name FROM stu...	1000 row(s) returned	0.062 sec / 0.000 sec
35	19:23:11	SELECT AVG(salary) AS mean_salary FRO...	Error Code: 1146. Table 'dbaclab2025.teac...	0.000 sec
36	19:23:29	SELECT AVG(salary) AS mean_salary FRO...	1 row(s) returned	0.000 sec / 0.000 sec

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench

myuserco x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- dept_name
- tot_cred
- Indexes
 - PRIMARY
 - student_fk1
- Foreign Keys
 - student_fk1
- Triggers
- student01
- takes
 - Columns
 - ID
 - course_id

Administration Schemas

Information

Column: grade

Collation: utf8mb4_0900_ai_ci

Definition:

grade varchar(2)

Object Info Session

Query 1 x DDL_createtbwithpredrop_al largeRelationsInsertFile

Limit to 1000 rows

SQLAdditions

Jump to

61 SET @total_count := (SELECT COUNT(*) FROM instructor);

62

63 -- 计算中位数（假设记录数为奇数时，取中间值；偶数时，取

64 SELECT

65 CASE

66 WHEN @total_count % 2 = 1 THEN

Result Grid

median_salary
80332.390000

Result 12 x Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
35	19:23:11	SELECT AVG(salary) AS mean_salary FRO...	Error Code: 1146. Table 'dbaclab2025.teac...	0.000 sec
36	19:23:29	SELECT AVG(salary) AS mean_salary FRO...	1 row(s) returned	0.000 sec / 0.000 sec
37	19:24:42	SELECT PERCENTILE_CONT(0.5) WITHI...	Error Code: 1064. You have an error in your...	0.000 sec
38	19:29:03	SET @total_count := (SELECT COUNT(*) ...	0 row(s) affected	0.016 sec
39	19:29:03	SELECT CASE WHEN @total_co...	1 row(s) returned	0.000 sec / 0.000 sec

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

代码

sql

```

#计算平均值
SELECT AVG(salary) AS mean_salary FROM instructor;

#计算中位数
-- 计算总记录数
SET @total_count := (SELECT COUNT(*) FROM instructor);

-- 计算中位数（假设记录数为奇数时，取中间值；偶数时，取中间两个值的平均值）
SELECT
    CASE
        WHEN @total_count % 2 = 1 THEN
            (SELECT salary
             FROM (SELECT salary, @rownum := @rownum + 1 AS rownum
                   FROM instructor, (SELECT @rownum := 0) r
                   ORDER BY salary) sub
             WHERE sub.rownum = (@total_count + 1) / 2)
        ELSE
            (SELECT AVG(salary)
             FROM (SELECT salary, @rownum := @rownum + 1 AS rownum
                   FROM instructor, (SELECT @rownum := 0) r
                   ORDER BY salary) sub
             WHERE sub.rownum IN (@total_count / 2, (@total_count
 / 2) + 1))
    END AS median_salary;

```

6. 分析两个 SQL 语句的区别，主要看 JOIN 条件位置。

语句不同。a) 中 ON 条件是 student.ID = takes.ID，正确关联；b) 中 ON true，然后 WHERE student.ID = takes.ID，会导致 LEFT JOIN 变成 INNER JOIN 效果，因为 WHERE 过滤了 null，丢失左表不匹配的数据。

a 语句：ON student.ID = takes.ID 正确关联两表，保留左表（student）所有记录。

b 语句：ON true 使连接无过滤，WHERE student.ID = takes.ID 会过滤掉左表不匹配的 null，实际效果等同于内连接（丢失左表未匹配数据）。

实验结论

通过本次实验，熟练掌握了 SQL 的核心操作，包括复杂查询、多表连接及聚合函数的应用。实验结果验证了关系型数据库的逻辑设计原则，为后续高级 SQL 技术（如事务处理、索引优化）奠定了基础。